1. HAFTA – Git ve GitHub

Araştırma

1.Git Nedir?

-Git, yazılım geliştirme süreçlerinde kod değişikliklerini zaman içinde takip etmek, yönetmek ve farklı kişilerin aynı proje üzerinde eş zamanlı çalışmasını sağlamak için kullanılan dağıtık bir Versiyon Kontrol Sistemidir (VCS).Git, birçok proje için popüler bir VCS aracıdır ve özellikle açık kaynaklı projelerde sıkça kullanılır. Git, merkezi bir sunucuya gerek duymadan birçok kişinin aynı kod deposunu paylaşmasına izin verir. Ayrıca, kodun farklı sürümlerini veya dallarını oluşturarak, farklı geliştirme yolları üzerinde çalışmalarını sağlar. Bu, yazılım geliştirme sürecindeki işbirliğini ve verimliliği artırır.(https://github.com/mkdemir/Git_ve_Github_Rehberi#git-nedir)

GitHub nedir?

-GitHub, Git ile takip edilen yazılım projelerini depolamak, yönetmek, paylaşmak ve başkalarıyla ortaklaşa geliştirmek için kullanılan web tabanlı bir platformdur.

Microsoft'a ait olan GitHub, dünyanın en büyük açık kaynak kodlu yazılım geliştirici topluluklarından biridir.(https://github.com/mkdemir/Git_ve_Github_Rehberi#github-nedir)

Git ve GitHub Arasındaki Farklar nelerdir?

-Git, sadece bir sürüm kontrolü sistemidir ve yerel bir bilgisayarda çalışırken GitHub, Git üzerine inşa edilmiş bir depolama sistemidir.

Git, bir projenin sürüm kontrolünü yönetirken, GitHub, projenin paylaşımı, işbirliği, sorun takibi ve kod incelemesi gibi ek özellikler sunar.

Git,verileri yerel bilgisayarda depolar.Github, verileri GitHub sunucularında depolar.

Git,İnternete ihtiyaç duymadan çalışır. Github ise internet üzerinden erişim gerektirir.(https://github.com/TuncerKARAARSLAN-VB/training-kit-git-git-workflow-sifirdan-zirveye/blob/main/01-git-tarihcesi/01-04-git-github-arasindaki-farklar.md)

Repository Nedir?

-Repository (depo), bir projenin kaynak kodlarının, dosyalarının ve bu dosyalara ait tüm değişiklik geçmişinin saklandığı özel bir klasördür.(https://vigo.github.io/git-puf-noktalari/bolum-01/03-repository-nedir/)

Commit nedir?

-Yazılım geliştirme ve veri yönetiminde yapılan değişiklikleri kalıcı hale getirmek veya bir sistemin o andaki durumunu anlık görüntü olarak kaydetmek anlamına gelir. (https://www.techcareer.net/blog/commit-nedir-ne-ise-yarar)

Branch nedir?

-Ana kod deposundan ayrılarak üzerinde bağımsız değişiklikler yapabileceğiniz çalışma kolu anlamına gelir. Yazılımcılar ana koda zarar vermeden yeni bir özelliği bu kolda geliştirir ve işleri bittiğinde ana koda eklerler.(https://ismetkizgin.medium.com/branch-nedir-nas%C4%B1l-kullan%C4%B1l%C4%B1r-github-c6a58d3d4d84)

Merge nedir?

-Farklı dallarda (branch) yapılan kod değişikliklerini tek bir dalda birleştirme işlemidir. Örneğin, yeni bir özellik eklemek için açtığınız dalı, projenin ana dal ile bu komutla birleştiririz.(https://www.git-tower.com/learn/git/faq/git-list)

Clone nedir?

-Uzaktaki bir sunucuda (örneğin GitHub'da) duran bir projenin tüm geçmişi ve dosyalarıyla birlikte bir kopyasını kendi bilgisayarınıza indirmektir. Sıfırdan bir projeye dahil olurken ilk yapılan işlemdir.Yani dosyanın bir kopyasını indirmektir.(https://coddy.tech/learn/tr/terminal/version_control/push_pull_and_clone)

Pull nedir?

-Uzaktaki sunucuda (GitHub'da) başkaları tarafından yapılan değişiklikleri kendi bilgisayarınıza indirmek ve kodlarınızı güncellemektir. Bir nevi projenizi "tazeleme" işlemidir.(https://medium.com/mindorks/what-is-git-commit-push-pull-log-aliases-fetch-config-clone-56bc52a3601c)

Push nedir?

-Kendi bilgisayarımızda yaptığımız kod değişikliklerini, projenin uzaktaki sunucusuna (GitHub'a) geri yüklemektir. Böylece yaptığımız değişiklikleri diğer arkadaşlarımız da görebilir.(https://medium.com/mindorks/what-is-git-commit-push-pull-log-aliases-fetch-config-clone-56bc52a3601c)

Gitignore nedir?

-Göz ardı edilen dosyalar denir, deponuzun kök dizininde kontrol edilen özel bir dosyada izlenir . Açık bir gitignore komutu yoktur: bunun yerine .gitignore, göz ardı etmek istediğiniz yeni dosyalarınız olduğunda dosya elle düzenlenmeli ve commit edilmelidir. .gitignore Dosyalar, göz ardı edilip edilmeyeceklerini belirlemek için deponuzdaki dosya adlarıyla eşleştirilen kalıplar içerir.(https://www.atlassian.com/git/tutorials/saving-changes/gitignore)

SEKTÖRLER

PROJE 1

-Proje adı: nadir-kitaplar.github.io

-Github adresi:https://github.com/nadir-kitaplar/nadir-kitaplar.github.io

-Kullanılan programlama dili: PYTHON, HTML

-Kullanılan framework: Framework kullanılmamıştır.

-Projenin Amacı: Nadir kitapların, çevirilerin veya eski metinlerin dijital ortama aktarılması, derlenmesi, korunması ve web sitesi üzerinden okuyuculara sunulması.

Yaklaşık Commit Sayısı: 2 commit

-Contributor Sayısı: Projede tek bir geliştiricinin katkısı göründüğü için 1 kişidir.

-Klasör Yapısı:

books: Kitap içeriklerinin tutulduğu klasör.

data: Projeye ait veri dosyaları.

previews: Önizleme görselleri veya sayfaları.

svg: Vektörel ikon ve çizimler.

Readme yapısı:proje başlığı ve genel amacın açıklandığı bir giriş, canlı web sitesi bağlantısı, öne çıkan eserin (Sanzo Wada Renk Sözlüğü) detayları, içerik/bölüm tablosu, terminal üzerinden hızlı yayınlama (CLI) komutları, klasör yapısı haritası ve telif/lisans bilgilerini içerecek şekilde hiyerarşik bir Markdown yapısıyla oluşturulmuştur.

PROJE 2

-Proje Adı: Firecraw

-GitHub Adresi: https://github.com/firecrawl/firecrawl

-Kullanılan Programlama Dili: TypeScript, JavaScript ve Python bileşenleri

-Kullanılan Framework: Node.js, Express / NestJS ve modern web/API yapıları.

-Projenin Amacı: İnternet sitelerindeki karmaşık sayfaları ve verileri okuyup temizleyerek, yapay zeka modellerinin ve uygulamalarının kolayca anlayabileceği düzenli metinlere dönüştüren açık kaynaklı bir web tarama aracıdır.

-Yaklaşık Commit Sayısı: 6.778 commit

-Contributor Sayısı: Yüzlerce katkıda bulunan geliştirici.(büyük bir açık kaynak projesidir).

-Klasör Yapısı:

github/ (GitHub iş akışları ve şablonlar)

apps/ (Ana uygulama ve servis dizinleri) examples/ (Örnek kullanım senaryoları) firecrawl-cli / firecrawl-cli-skills (Komut satırı araçları)

firecrawl-skills / firecrawl-workflows (Yapay zeka ve otomasyon iş akışları)

img/ (Proje görselleri ve logolar)

kills/ (Entegrasyon ve modül yetenekleri)

-README Yapısı: Firecrawl projesinin README.md dosyası; projenin sunduğu temel avantajları ve güvenilirlik oranlarını açıklayan "Why Firecrawl?" bölümüyle başlar. Ardından arama, kazıma (scrape), tarama (crawl) ve yapay zeka ajan özelliklerini detaylandıran "Feature Overview", farklı dillerdeki SDK kullanımlarını ve API uç noktalarını gösteren "Quick Start / SDKs" rehberi ile projeyi kendi sunucusunda çalıştırmak isteyenler için lisans ve kurulum yönlendirmelerini içeren bir hiyerarşiye sahiptir.

PROJE 3

-Proje Adı: KACHOW-Teknofest-2026

-GitHub Adresi: https://github.com/chyp3r/KACHOW-Teknofest-2026

-Kullanılan Programlama Dili: Python (Backend için) ve TypeScript / JavaScript (Frontend için) Kullanılan Framework: Backend tarafında FastAPI, LangGraph, LangChain; Frontend tarafında ise React kullanılmıştır.

-Projenin Amacı: Teknofest 2026 Doğal Dil Ajanları Yarışması kapsamında geliştirilen; kamu kurumlarındaki resmi evrakları yapay zeka ile analiz eden, doğrulayan, taslak üreten ve doğru birime yönlendiren çoklu yapay zeka ajanı tabanlı bir karar destek platformudur.

-Yaklaşık Commit Sayısı:815 commit

-Contributor Sayısı: Katkıda bulunan geliştirici/geliştiriciler (Topluluk projeleri ve yarışma ekibi tabanlı).

-Klasör Yapısı:

github/: CI/CD ve GitHub Actions iş akışları

assets/: Projeye ait medya dosyaları ve görseller backend/: FastAPI, LangGraph ajanları ve veritaba

datasets/: Mevzuat ve test veri setler

deploy/ & monitoring/: Docker, Kubernetes, Prometheus ve Grafana kurulum dosyaları

frontend/: React + TypeScript arayüz kodları

-README Yapısı: platformun vizyonunu ve amacını açıklayan "KACHOW nedir?" bölümüyle başlar. Devamında "Demo ve Arayüz", sistemin sunduğu özellikleri açıklayan "Temel Yetenekler" (evrak analizi, mevzuat arama, taslak üretimi, insan onayı vb.), Mermaid şemalarıyla desteklenmiş "Sistem Mimarisi" ve "AI İş Akışları" bölümleri yer alır.

Metnin orta ve alt kısımlarında multi-tenant güvenlik (RBAC/RLS), kuruma özel LoRA/DPO öğrenme süreçleri, yerel/sunucu çalışma modları (Ollama vs. Evren API), kapsamlı "Değerlendirme / Benchmark" (OCR, LLM Judge, Red Team, Performans) test sonuçları, "Hızlı Başlangıç (Docker/K8s)" kurulum adımları ve "Proje Klasör Yapısı" haritasını içeren profesyonel bir teknik dokümantasyona sahiptir.